

Objetivos da Aula:

- guardar informações em variáveis e conhecer os tipos da linguagem.
- organizar dados com arrays e objetos.
- entender o formato JSON e converter texto em dado.
- montar a eleição do projeto em arquivos pequenos e modulares.

De onde viemos e para onde vamos

Na Aula 1 nós fizemos:

instalamos o Node e conferimos a versão.
criamos o package.json com npm init.
declaramos "type": "module" para usar import e export.
instalamos express, dotenv e nodemon.
subimos um servidor respondendo em localhost.

Hoje o assunto é outro:

antes de servir dados pela rede, precisamos saber guardar dados.

Toda a eleição vai existir dentro do projeto: cargos, seções, candidatos e votos.

Ao final, um comando de terminal percorre tudo, conta os votos e apura o resultado.

Sem banco de dados e sem tela: só a linguagem.

O projeto modelo: a Urna Eletrônica

Painel do administrador:

roda no servidor e devolve HTML.
cadastrar os candidatos da eleição.
mostra a apuração dos votos.
é a parte acoplada do sistema.

A urna do eleitor:

roda no navegador e consome uma API.
teclado numérico de dois dígitos.
mostra a foto do candidato e registra o voto.
é a parte desacoplada do sistema.

Data Types in JS

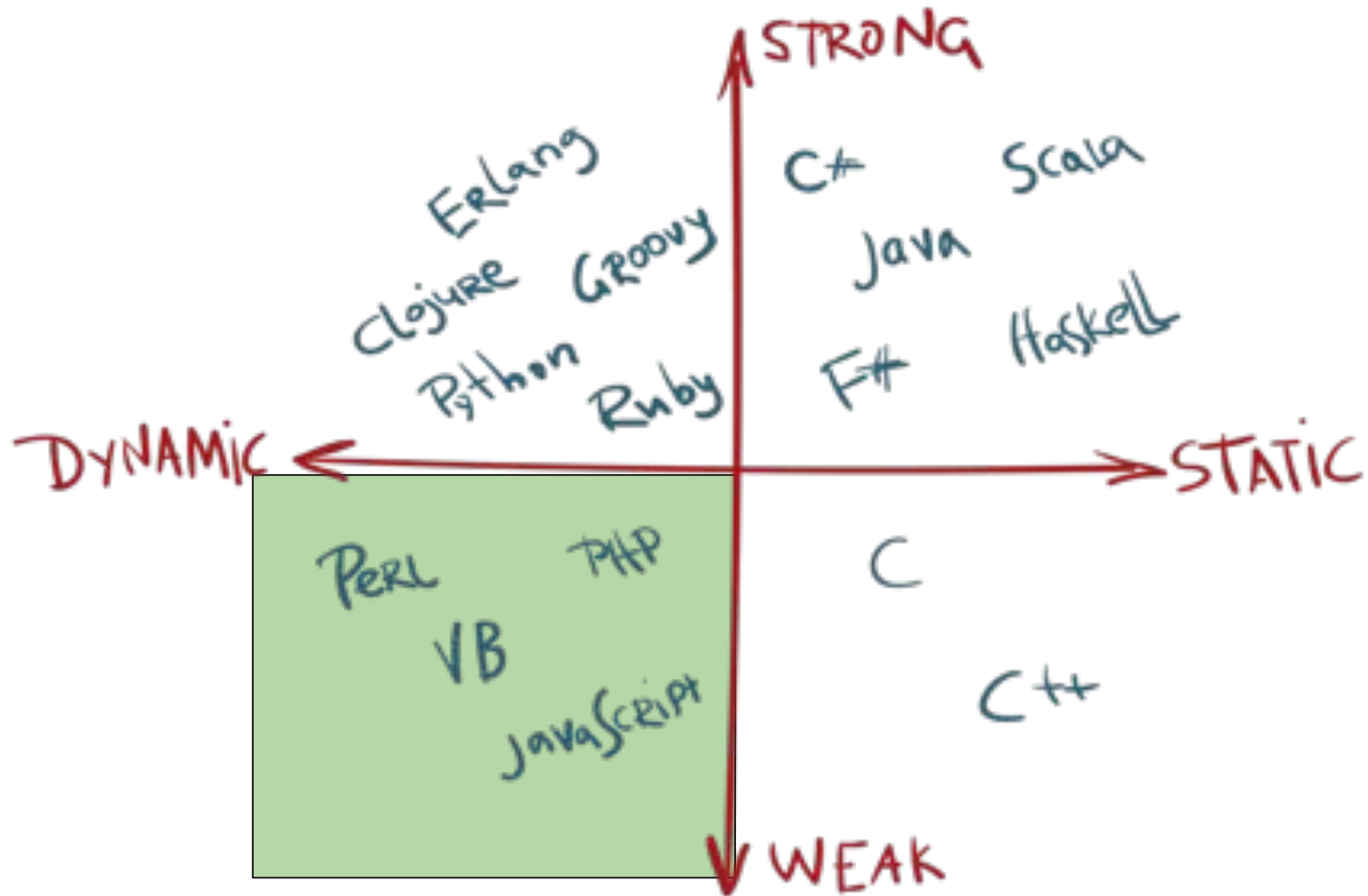
Primitive

String
Number
Boolean
Null
Undefined

Non-Primitive

Object
Array
Function

Tipo de dados



Valor e variável

```
// Variavel e um nome dado a um valor, para usar depois.
```

```
const nome = 'Rick Sanchez';  
const numeroUrna = '10';  
const totalDeVotos = 5;
```

```
// A partir daqui, escrever nome e o mesmo que escrever o texto,  
// com a vantagem de que o nome explica o que aquilo e.
```

```
console.log(nome);           // Rick Sanchez  
console.log(totalDeVotos);   // 5
```

const e let

const

cria um nome que não pode ser reatribuído.

É a escolha padrão do curso.

```
const tituloDaEleicao = 'Urna Eletronica';
```

Comece sempre com const. Troque para let só quando o próprio programa reclamar.

let

cria um nome que pode receber outro valor depois.

Use quando o valor muda mesmo.

```
let totalDeNominiais = 0;
```

```
totalDeNominiais = totalDeNominiais + 1;
```

var não usamos em código novo.

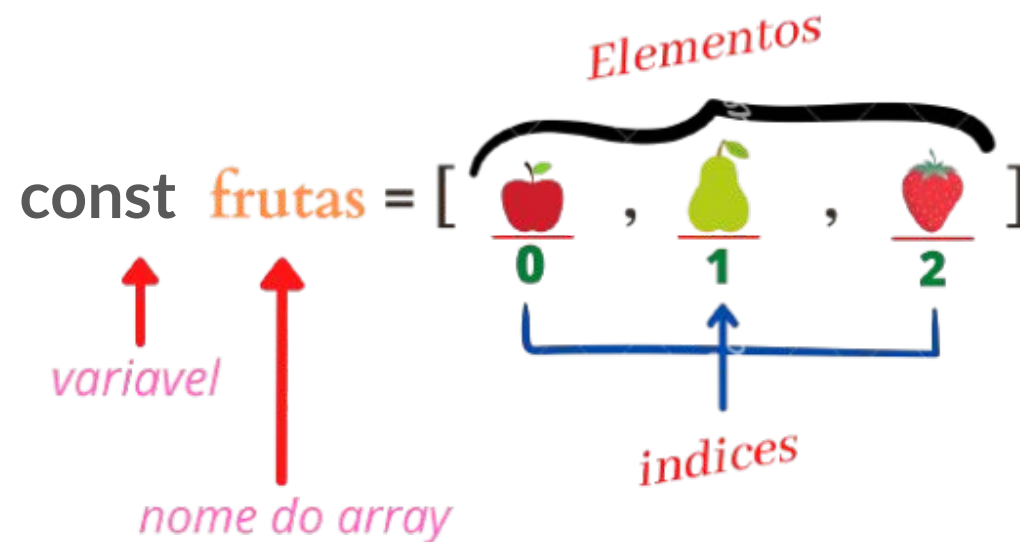
Ele tem regras de visibilidade confusas. Se aparecer, é código legado.

O que são Arrays? []

Um array é uma coleção ordenada de elementos.

como uma **fila** de caixas, onde cada caixa pode conter um item (que pode ser um número, uma string, outro array, um objeto, etc.).

Análise sintática de um array.

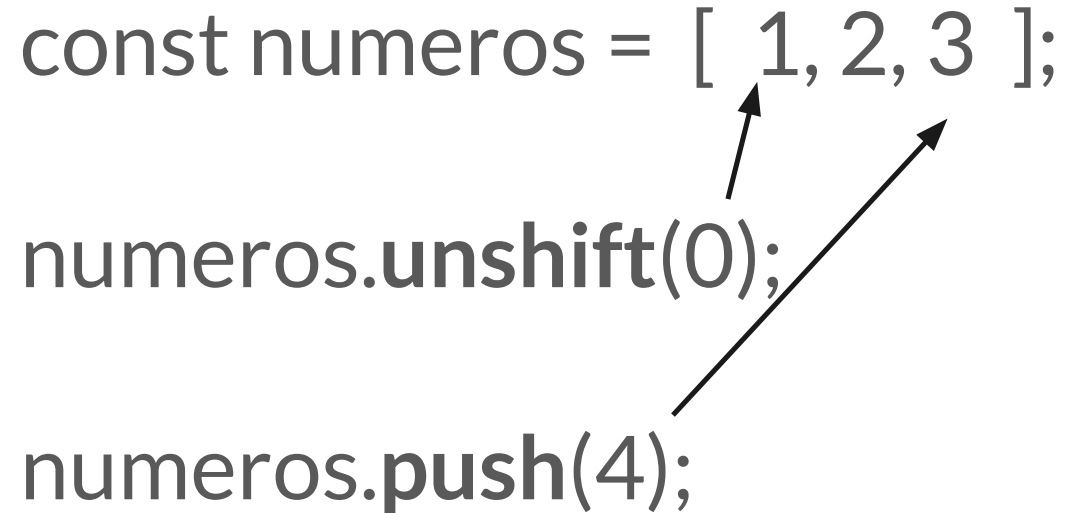


```
const frutas = ["maçã", "pêra", "morango"];
```


Manipulação de Arrays - Adicionar elementos

- **.push()**
Adiciona um ou mais elementos ao final do array.
- **.unshift()**
Insere um ou mais elementos no início do array.

```
const numeros = [ 1, 2, 3 ];  
numeros.unshift(0);  
numeros.push(4);
```



The diagram illustrates the effect of the `unshift` and `push` methods on the `numeros` array. The initial array is `[1, 2, 3]`. The `unshift(0)` method adds the value `0` to the beginning of the array. The `push(4)` method adds the value `4` to the end of the array. Arrows indicate the insertion points: one arrow points from `unshift(0)` to the space before the first element (`1`), and another arrow points from `push(4)` to the space after the last element (`3`).

Manipulação de Arrays - Remover elementos

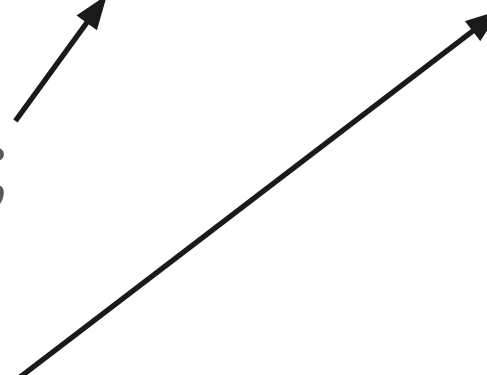
- **.pop()**
Remove o último elemento de um array e retorna esse elemento.
- **.shift()**
Remove o primeiro elemento do array e retorna esse elemento.

```
const numeros = [0, 1, 2, 3, 4];
```



```
numeros.shift();
```

```
numeros.pop();
```



O que são Objetos? { }

Os objetos são uma das estruturas de dados mais importantes e **versáteis**.

Eles permitem agrupar

dados = **propriedades**

comportamentos = **métodos**

```
const pessoa = {
```

```
  nome: "Maria",  
  idade: 28,
```

```
  falar: function() {  
    console.log("Olá, sou " + this.nome);  
  }  
};
```

O que são Objetos? { }

Atributos
coisas que tem

Comportamento
coisas que faz



```
const pessoa = {
```

```
  nome: "Maria",  
  idade: 28,
```

```
  falar: function() {  
    console.log("Olá, sou " + this.nome);  
  }  
};
```

Declarando um Objeto

Dentro das chaves { } se define as propriedades e métodos, muitas vezes apenas propriedades. Diferente dos arrays as chaves ou posições são explícitas

```
const pessoa = {  
  nome: "Maria",  
  idade: 28,  
  falar: function() {  
    console.log("Olá, sou " + this.nome);  
  }  
};
```

Acessando Propriedades

Notação de Ponto:

pessoa.nome

retornará "Maria".

Notação de Colchetes:

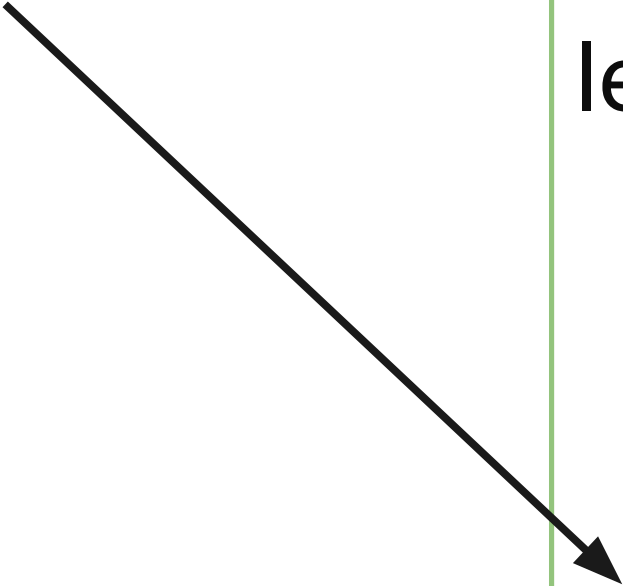
pessoa['idade']

retornará 28.

```
const pessoa = {  
  nome: "Maria",  
  idade: 28,  
  falar: function() {  
    console.log("Olá, sou " + this.nome);  
  }  
};
```

Adicionando nova Propriedade

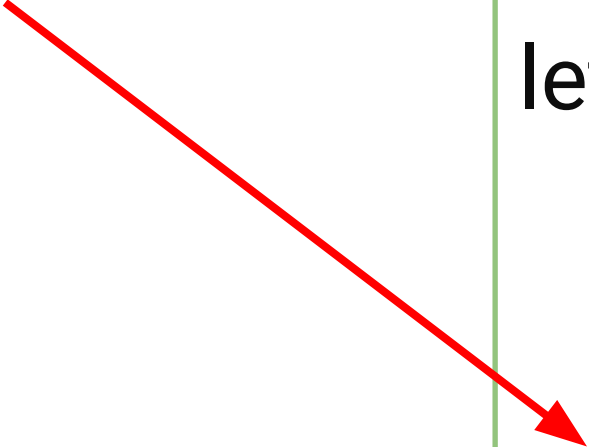
```
pessoa.profissao = "Desenvolvedora";
```



```
let pessoa = {  
  nome: "Maria",  
  idade: 28  
};
```

Modificando Propriedades

```
pessoa.idade = "30";
```



```
let pessoa = {  
  nome: "Maria",  
  idade: 28,  
  profissao : "Desenvolvedora"  
};
```


Objetos e Arrays Combinados

Em desenvolvimento web, encontramos situações com arrays de objetos, como no uso de formulários ou dados que consumimos do servidor para apresentar no frontend.

```
let livros = [  
  { titulo: "PHP", autor: "ABCD", ano: 1995 } ,  
  { titulo: "Javascript", autor: "A1B2", ano: 1995 } ,  
  { titulo: "Python", autor: "XYZ", ano: 1991 }  
];
```

Para acessar Javascript = livros[1].titulo



Entrada de dados

```
import * as readline from 'node:readline/promises';  
import { stdin as input, stdout as output } from 'node:process';  
  
const rl = readline.createInterface({ input, output });  
const name = await rl.question("Qual o seu nome ");  
console.log(`Olá, ${name}!`);  
rl.close();
```

Atenção: const impede reatribuir, e não impede alterar

```
const lista = ['10', '20'];
```

```
lista.push('30');          // funciona: alterou o CONTEUDO  
console.log(lista);        // [ 10, 20, 30 ]
```

```
lista = ['99'];            // erro: tentou trocar o VALOR do nome
```

```
// Saída real no terminal:  
// TypeError: Assignment to constant variable.
```

```
// O mesmo vale para objeto:  
const obj = { nome: 'Rick' };  
obj.nome = 'Morty';        // funciona, sem reclamar
```

```
// const protege a etiqueta, não o conteúdo da caixa.
```

Tipagem dinâmica e typeof

// A variavel nao declara o tipo: ela assume o tipo do valor.

```
let coisa = 'texto';  
coisa = 42;           // permitido, sem erro nenhum
```

```
const nome = 'Rick Sanchez';  
const totalDeVotos = 5;  
const ativo = true;  
let mensagem;
```

```
console.log(typeof nome);           // string  
console.log(typeof totalDeVotos);  // number  
console.log(typeof ativo);         // boolean  
console.log(typeof mensagem);      // undefined  
console.log(typeof null);          // object <- pegadinha historica
```

Os cinco tipos que este projeto usa

string

texto, entre aspas ou crases. Ex: nome, numeroUrna, identificador.

number

número inteiro ou decimal. Ex: id, totalDeVotos, percentual.

boolean

true ou false. Ex: ativo, aberta.

undefined

a variável existe e ninguém colocou nada dentro. É o esquecimento.

null

ausência intencional de valor. É a decisão.

No nosso domínio a diferença é concreta:

um voto em branco tem candidatold igual a null, porque a ausência do candidato É a informação.

Decisão de projeto: número de urna é texto

```
// Saida real no terminal:
```

```
// Number("07") -> 7          o zero sumiu
```

```
// "07" === "7"  -> false
```

```
// Como numero, 07 e 7 sao o mesmo valor.
```

```
// Como texto, "07" e "70" sao candidatos diferentes,
```

```
// que e o que a urna precisa.
```

```
// A regra geral, para levar para a carreira:
```

```
// se o zero a esquerda importa, o dado e TEXTO.
```

```
// Vale para CEP, CPF, matricula, agencia bancaria
```

```
// e numero de urna.
```

Template strings: a ferramenta do curso inteiro

```
const nome = 'Rick Sanchez';
const numeroUrna = '10';

// Sem template string: concatenacao com sinal de mais.
const antiga = '0 candidato ' + nome + ' tem o numero ' + numeroUrna;

// Com template string, entre crases, e interpolacao com ${}:
const linha = `0 candidato ${nome} tem o numero ${numeroUrna}`;

// Ela tambem aceita quebra de linha sem truque nenhum:
const relatorio = `Eleicao: Urna Eletronica
Candidato: ${nome}
Numero:    ${numeroUrna}`;

// Dentro das chaves cabe qualquer expressao:
console.log(`Total: ${3 + 1 + 1}`);
```

Comparação: use sempre o operador estrito

```
// Saida real no terminal:
```

```
// 10 == "10"    -> true    compara so o valor, convertendo o tipo  
// 10 === "10"  -> false   compara valor E tipo  
// 0 == ""      -> true    e isto quase nunca e o que voce queria
```

```
// Regra da nossa sala: sempre === e !==
```

```
// Operadores logicos para combinar condicoes:
```

```
// && (E)    voto.tipo === 'nominal' && voto.candidatoId === 1  
// || (OU)   secao === '0001' || secao === '0002'  
// ! (NAO)   !secao.aberta
```


truthy e falsy: saída real do terminal

```
false      -> falsy
0           -> falsy
           -> falsy      (texto vazio)
null        -> falsy
undefined   -> falsy
NaN         -> falsy

"0"         -> truthy      <- texto com conteudo
[]          -> truthy      <- array vazio e truthy
```

Sao seis os valores falsy. Todo o resto e truthy.

Cuidado: `if (lista)` nao pergunta se a lista tem itens,
pergunta se ela existe. Para saber se tem itens: `lista.length > 0`

Decisão: if, else e o ternário

```
const voto = { tipo: 'branco', candidatoId: null };
```

```
if (voto.tipo === 'nominal') {  
  console.log('Voto em um candidato');  
} else if (voto.tipo === 'branco') {  
  console.log('Voto em branco');  
} else {  
  console.log('Voto nulo');  
}
```

// Para decisoes curtas que produzem UM VALOR, a forma compacta:

```
const situacao = candidato.ativo ? 'ativo' : 'desativado';  
const estado = secao.aberta ? 'ABERTA' : 'FECHADA';
```

Array: lista ordenada, começando do zero

```
const numeros = ['10', '20', '30'];
```

```
console.log(numeros[0]);      // 10
```

```
console.log(numeros[2]);      // 30
```

```
console.log(numeros.length);  // 3
```

```
console.log(numeros[9]);      // undefined, e nao erro
```

```
// Tres detalhes que geram duvida na primeira semana:
```

```
// 1. o primeiro item e a posicao 0, o ultimo e length - 1
```

```
// 2. pedir posicao que nao existe NAO da erro, devolve undefined
```

```
// 3. length e a quantidade, nao o ultimo indice
```

```
numeros.push('40');          // acrescenta no final
```

Repetição: for e for...of

```
const numeros = ['10', '20', '30'];
```

```
// for classico: inicializacao, condicao e incremento.
```

```
for (let i = 0; i < numeros.length; i = i + 1) {  
  console.log(`posicao ${i}: ${numeros[i]}`);  
}
```

```
// for...of: percorre os VALORES, sem contador.
```

```
for (const numero of numeros) {  
  console.log(numero);  
}
```

```
// Prefira o for...of: ele diz o QUE voce quer,  
// em vez de COMO fazer. Menos pecas, menos erro.
```

Objeto literal: dados com nome

```
const candidato = {  
  id: 1,  
  nome: 'Rick Sanchez',  
  numeroUrna: '10',  
  imageUrl: 'https://rickandmortyapi.com/api/character/avatar/1.jpeg',  
  cargoId: 1,  
  ativo: true,  
};  
  
// Notacao de ponto: quando voce sabe o nome da propriedade.  
console.log(candidato.nome);           // Rick Sanchez  
  
// Notacao de colchetes: quando o nome esta numa variavel.  
const campo = 'numeroUrna';  
console.log(candidato[campo]);         // 10
```

Array de objetos: o padrão central do projeto

```
const candidatos = [  
  { id: 1, nome: 'Rick Sanchez', numeroUrna: '10', ativo: true },  
  { id: 2, nome: 'Morty Smith', numeroUrna: '20', ativo: true },  
  { id: 3, nome: 'Summer Smith', numeroUrna: '30', ativo: true },  
];
```

```
// Percorrer e buscar: isto é, em essência, o que a API  
// vai fazer quando o eleitor digitar dois números.
```

```
const numeroDigitado = '20';  
let encontrado = null;
```

```
for (const candidato of candidatos) {  
  if (candidato.numeroUrna === numeroDigitado) {  
    encontrado = candidato;  
  }  
}
```

Desestruturação e espalhamento

```
const candidato = { id: 1, nome: 'Rick Sanchez', numeroUrna: '10', ativo: true };
```

```
// Desestruturacao: extrai propriedades para variaveis.
```

```
const { nome, numeroUrna, ativo } = candidato;
```

```
console.log(nome, numeroUrna, ativo); // Rick Sanchez 10 true
```

```
// Aceita valor padrao para propriedade ausente:
```

```
const { partido = 'sem partido' } = candidato;
```

```
// Espalhamento: copia mudando so uma propriedade.
```

```
const copia = { ...candidato, ativo: false };
```

```
console.log(copia.ativo); // false
```

```
console.log(candidato.ativo); // true, o original ficou intacto
```

```
// Atencao: o que vem DEPOIS vence. A ordem importa.
```

JSON: é texto, não é objeto

O que muda em relação ao objeto:

nome da propriedade sempre entre aspas duplas.

aspas de texto sempre duplas.

vírgula depois do último item é proibida.

comentário não existe.

função como valor não existe.

Por que este formato domina:

é texto, então atravessa qualquer rede e cabe em qualquer arquivo.

é legível por humanos.

é entendido por Python, Java, C#, PHP e todas as outras.

Você já o viu no package.json da Aula 1, e vai reencontrá-lo em todas as respostas do servidor.

JSON.parse e JSON.stringify

```
const candidato = { id: 1, nome: 'Rick Sanchez', numeroUrna: '10' };
```

```
// De objeto para texto.
```

```
const comoTexto = JSON.stringify(candidato);
```

```
console.log(comoTexto);
```

```
// {"id":1,"nome":"Rick Sanchez","numeroUrna":"10"}
```

```
console.log(typeof comoTexto); // string
```

```
// De texto de volta para objeto.
```

```
const deVolta = JSON.parse(comoTexto);
```

```
console.log(deVolta.nome); // Rick Sanchez
```

```
console.log(typeof deVolta); // object
```

```
// stringify transforma EM TEXTO. parse INTERPRETA o texto.
```

Mãos à obra: por que arquivos pequenos

A estrutura de hoje:

src/dados/cargos.js

src/dados/secoes.js

src/dados/candidatos.js

src/dados/votos.js

src/dados/index.js

scripts/dadosDemo.js

Um arquivo, um assunto.

Quem procura os candidatos abre candidatos.js, e não rola trezentas linhas.

Arquivo pequeno se lê inteiro.

É entre os pedaços de um arquivo grande que os defeitos moram.

Arquivo pequeno reduz conflito em equipe.

Duas pessoas em arquivos diferentes não se atropelam.

src/dados/candidatos.js

```
// src/dados/candidatos.js
// Os candidatos da eleicao.

export const candidatos = [
  {
    id: 1,
    nome: 'Rick Sanchez',
    numeroUrna: '10',
    imageUrl: 'https://rickandmortyapi.com/api/character/avatar/1.jpeg',
    cargoId: 1,
    ativo: true,
  },
  // ... Morty Smith com o numero 20 e Summer Smith com o 30
];

// cargoId guarda o id do cargo, e nao o nome dele.
// ativo existe porque candidato nao se apaga: desativa-se.
```

```
export const candidatos = [  
  {  
    id: 1,  
    nome: 'Rick Sanchez',  
    numeroUrna: '10',  
    imageUrl: 'https://rickandmortyapi.com/api/character/avatar/1.jpeg',  
    cargoId: 1,  
    ativo: true,  
  },  
  {  
    id: 2,  
    nome: 'Morty smith',  
    numeroUrna: '20',  
    imageUrl: 'https://rickandmortyapi.com/api/character/avatar/2.jpeg',  
    cargoId: 1,  
    ativo: true,  
  },  
  {  
    id: 3,  
    nome: 'Summer smith',  
    numeroUrna: '30',  
    imageUrl: 'https://rickandmortyapi.com/api/character/avatar/3.jpeg',  
    cargoId: 1,  
    ativo: true,  
  }  
];
```

src/dados/votos.js e a porta única

```
// src/dados/votos.js  
// Voto branco e nulo não tem candidato: candidatoId é null.
```

```
export const votos = [  
  { id: 1, tipo: 'nominal', candidatoId: 1, secaoId: 1 },  
  { id: 2, tipo: 'nominal', candidatoId: 1, secaoId: 1 },  
  { id: 3, tipo: 'nominal', candidatoId: 2, secaoId: 1 },  
  { id: 4, tipo: 'branco', candidatoId: null, secaoId: 1 },  
  { id: 5, tipo: 'nulo', candidatoId: null, secaoId: 1 },  
];
```

```
// src/dados/index.js -> a porta única dos dados
```

```
export { candidatos } from './candidatos.js';  
export { votos } from './votos.js';
```

scripts/dadosDemo.js: contando os votos

```
import { candidatos, votos } from '../src/dados/index.js';

let nominais = 0;
let brancos = 0;
let nulos = 0;

for (const voto of votos) {
  if (voto.tipo === 'nominal') {
    nominais = nominais + 1;
  } else if (voto.tipo === 'branco') {
    brancos = brancos + 1;
  } else {
    nulos = nulos + 1;
  }
}

// Os tres contadores sao let, e nao const:
// o valor muda a cada volta do laco.
```

A apuração: um laço dentro do outro

```
for (const candidato of candidatos) {  
  let recebidos = 0;  
  
  for (const voto of votos) {  
    if (voto.tipo === 'nominal' && voto.candidatoId === candidato.id) {  
      recebidos = recebidos + 1;  
    }  
  }  
  
  const percentual = votos.length === 0  
    ? 0  
    : (recebidos / votos.length) * 100;  
  
  console.log(`${candidato.numeroUrna} ${candidato.nome} ${recebidos}`);  
}  
  
// O ternario protege a divisao por zero: 0 / 0 produz NaN.
```

```
import { candidatos, votos } from '../src/dados/index.js';
let nominais = 0;
let brancos = 0;
let nulos = 0;
for (const voto of votos) {
  if (voto.tipo === 'nominal') {
    nominais = nominais + 1;
  } else if (voto.tipo === 'branco') {
    brancos = brancos + 1;
  } else {
    nulos = nulos + 1;
  }
}
for (const candidato of candidatos) {
  let recebidos = 0;
  for (const voto of votos) {
    if (voto.tipo === 'nominal' && voto.candidatoId === candidato.id) {
      recebidos = recebidos + 1;
    }
  }

  const percentual = votos.length === 0 ? 0 : (recebidos / votos.length) * 100;
  console.log(`${candidato.numeroUrna} ${candidato.nome} ${recebidos} Porcentual ${percentual}%`);
}
console.log(`Brancos ${brancos} Nulos ${nulos}`);
```



```
node .\scripts\dadosDemo.js
```

```
10 Rick Sanchez 2 Porcentual 40%  
20 Morty smith 1 Porcentual 20%  
30 Summer smith 0 Porcentual 0%  
Branco 1 Nulos 1
```

Para praticar

- Crie um objeto para descrever um livro e imprima o typeof de cada propriedade.
- Acrescente uma terceira seção e um quarto candidato, e rode de novo sem tocar no script.
- Teste no terminal: `"10" === 10`, `Number("07") === 7`, `Boolean("0")`, `null == undefined`.
- Escreva o texto JSON de um voto, converta com parse, altere e converta de volta com stringify.
- Provoque o erro: ponha uma vírgula depois do último campo de um JSON e leia a mensagem.

Referências

PINHO, Diego Martins de. ECMAScript 6: entre de cabeça no futuro do JavaScript. Casa do Código, 2017. Atualizado em 05/2018.

PEREIRA, Caio Ribeiro. Livro de NodeJS. Casa do Código, 2013. Atualizado em 09/2021.

MDN Web Docs. Declarações e variáveis, Tipos de dados, Template strings, Array, Trabalhando com objetos, Laços e iterações, Trabalhando com JSON.

<https://developer.mozilla.org/pt-BR/>

W3Schools. JavaScript Data Types e JavaScript Arrays.

<https://www.w3schools.com/js/>